

RISC-V Based SoC for Hardware Performance Monitoring and Event Analysis

Complete Project Documentation

A memory-mapped Cycle Counter, Instruction Counter, and Performance Event Counter integrated around a faculty-provided RISC-V processor core, for RTL-based system performance observability and UART-based reporting.

Document Revision: 1.0 (Draft — Architecture Specification)
Status: Pre-implementation / Pre-simulation

List of Tables

Table 1	SoC Block Summary
Table 2	Top-Level Memory Map
Table 3	External Signals — Chip Boundary I/O
Table 4	System Bus Signals — Signal Breakdown
Table 5	Cycle Counter Registers
Table 6	Instruction Counter Registers
Table 7	Performance Event Counter Registers
Table 8	Event Select Encoding
Table 9	UART Registers
Table 10	Timer Registers
Table 11	GPIO Registers
Table 12	Core-Facing Bus/Event Signals (to be finalized against the provided core)
Table 13	RTL Implementation Order
Table 14	Unit Test Plan Summary
Table 15	Integration Test Plan
Table 16	Interrupt / Status Summary

1. Introduction

1.1 Background and Motivation

Software running on a processor can measure its own performance by reading standard RISC-V hardware performance counters — **mcycle** and **minstret** — through dedicated CSR (Control and Status Register) instructions. These counters are mandated to be accessible only to the hart itself: there is no path for an external bus master, a debug host, or a piece of simulation infrastructure to read them without the processor executing a CSR read and then storing the result to memory or a peripheral. Every such read therefore consumes at least one additional instruction slot, is subject to pipeline and CSR-access latency, and cannot be triggered from outside the processor's own instruction stream.

This creates two practical limitations for SoC-level performance analysis. First, software-only measurement adds execution overhead and can perturb the very activity being measured, particularly for short code regions. Second, the CSR interface exposes only what the ISA already standardizes (cycles and retired instructions); it does not provide a lightweight, extensible way to observe additional processor or system events — such as branch outcomes, memory-access activity, or bus-stall cycles — without modifying the processor's CSR file itself, which is often outside the scope of a project that is given a fixed, faculty-provided RISC-V RTL core.

A compact set of dedicated, memory-mapped hardware IP blocks addresses both limitations. By mirroring cycle and instruction-retirement activity in bus-visible registers and adding a third, configurable event counter, the processor — or any other bus master, such as a verification harness — can enable, clear, and read performance information as an ordinary memory-mapped access, with no CSR instruction, no dependency on core-internal counter logic, and no perturbation of the CSR pipeline stage.

1.2 Problem Statement

- 1 Modern SoC designs need a way to observe processor activity and evaluate system performance during program execution, but software-only measurement (CSR reads of mcycle/minstret) adds execution overhead and offers no direct, bus-level view of processor activity to external observers.
- 2 The standard RISC-V performance-counter CSRs expose only cycle count and retired-instruction count. There is no standardized, lightweight mechanism to count additional processor or system events (branches, memory accesses, bus stalls, etc.) without core RTL modification, which this project's faculty-provided RISC-V core is not expected to undergo.

1.3 Proposed Solution

This project addresses both problems by designing and verifying a RISC-V based SoC that:

- Uses the provided RISC-V RTL processor, unmodified, as the SoC's central processing unit.
- Integrates a dedicated **Cycle Counter** IP that increments on every enabled processor clock cycle and exposes the accumulated count through memory-mapped registers, independent of the CSR pipeline.
- Integrates a dedicated **Instruction Counter** IP that increments on every instruction-completion (retirement) event observed on the processor interface, giving a bus-visible equivalent of minstret.

- Integrates a dedicated, programmable **Performance Event Counter** IP that counts one selected processor or system event chosen through a small event-select register, extending monitoring beyond the two ISA-mandated counters without touching the core RTL.
- Connects the processor, Instruction Memory, Data Memory, UART, Timer, GPIO, and the three monitoring IPs through a single memory-mapped system bus with straightforward address decoding.
- Runs a small test application that clears, enables, and later reads the monitoring registers, and reports the collected performance information through UART.
- Is verified through RTL simulation covering processor bus accesses, address decoding, counter operation, control-register behaviour, and UART reporting.

1.4 Design Objectives

- 1 Integrate the provided RISC-V RTL processor as the SoC's central control unit without RTL modification to the core itself.
- 2 Implement a Cycle Counter IP with enable, clear, and 64-bit (two 32-bit register) accumulated-count read-back, tracking every enabled processor clock cycle.
- 3 Implement an Instruction Counter IP with enable, clear, and 64-bit accumulated-count read-back, tracking every instruction-retirement event supplied by the processor interface.
- 4 Implement a Performance Event Counter IP with enable, clear, an event-select register, and a 32-bit count register, supporting at least four selectable system/processor events.
- 5 Implement a memory-mapped system bus with straightforward, priority-free single-master address decoding connecting the processor to all peripherals and monitoring IPs.
- 6 Implement UART for reporting of measured performance values to a simulation console or host.
- 7 Implement GPIO for a simple completion or pass/fail status indicator.
- 8 Implement Timer as a mandatory baseline peripheral usable for test scheduling.
- 9 Produce a complete, aligned memory map covering Instruction Memory, Data Memory, and every peripheral and monitoring IP.
- 10 Develop a small test application that exercises a known computational task, configures the monitoring counters, and reports the resulting cycle, instruction, and event counts.
- 11 Verify the design through RTL simulation: bus access correctness, address decoding, counter accuracy, control-register behaviour, and UART reporting.

1.5 Scope

This document covers the complete architecture of the SoC: the processor core and its bus interface, the system bus interconnect and address decoder, the three performance-monitoring IP blocks, the mandatory UART/Timer/GPIO peripherals, the memory subsystem, the memory map, the register-level interface of every block, the control and data flow, the RTL design methodology, and the verification and test plan.

***Note:** Physical timing closure, FPGA bitstream generation, and silicon area/power figures are outside the scope of this project. The design targets technology-independent, synthesisable RTL, simulated at RTL level.*

1.6 Document Organisation

- Chapter 2: Technical Background — RISC-V performance-counter conventions, the provided processor core's assumed interface, and the memory-mapped bus protocol used.
- Chapter 3: Proposed SoC Architecture — block diagram, block definitions, system overview.
- Chapter 4: Detailed Description of Each SoC Block — functional description, interface, and role of every block.
- Chapter 5: Memory Subsystem and Memory Map — instruction/data memory and the complete address map.
- Chapter 6: Bus and Interconnect — bus protocol, signal handshake, address decoder.
- Chapter 7: Register-Level Interface — full register definitions for every monitoring IP and peripheral.
- Chapter 8: Control Flow — boot/initialisation, counter configuration, and readout sequencing.
- Chapter 9: Data Flow — cycle-counting, instruction-counting, event-counting, and UART reporting paths.
- Chapter 10: Design Methodology — RTL coding guidelines and implementation order.
- Chapter 11: Verification and Test Plan — test strategy, scenarios, and expected results.
- Chapter 12: Limitations and Future Enhancements.
- Chapter 13: Conclusion.
- References.

2. Technical Background

2.1 RISC-V Performance-Counter Conventions

The RISC-V privileged and unprivileged specifications define a small set of standard hardware performance counters accessible via CSR instructions: **cycle/cycleh** (mirrors of mcycle), which count core clock cycles, and **instret/instreth** (mirrors of minstret), which count retired instructions. Machine-mode software can additionally read and write the underlying mcycle/minstret CSRs directly, and a small number of implementation-defined event counters (mhpcounter3–31, paired with mhpmevent3–31) may be provided by a given core to count microarchitectural events such as branch mispredictions or cache misses.

Two properties of this CSR-based scheme motivate the present project. First, every one of these counters is read through a CSR instruction executed by the hart itself — there is no architectural path for an external bus master to observe them directly. Second, whether a given core actually implements the optional mhpcounter/mhpmevent pairs is entirely implementation-defined; a faculty-provided educational core is not guaranteed to expose any of them. This project does not rely on the provided core implementing mhpcounter/mhpmevent; instead, it builds independent, bus-attached counter IPs that observe the processor's external interface signals (clock enable, instruction-retire indication, and other bus activity) rather than the processor's internal CSR file.

2.2 The Provided RISC-V Processor Core

This project is built around a faculty-provided RISC-V RTL processor core. The architecture in this document assumes a conventional single-issue, in-order core exposing, at minimum: an instruction-fetch interface to Instruction Memory, a load/store interface to Data Memory and memory-mapped peripherals, a system clock and synchronous reset, and — for the purposes of the Instruction Counter and Performance Event Counter — an externally observable retirement or instruction-completion indication (for example, a single-cycle pulse asserted whenever an instruction commits its architectural state).

***Note:** The exact signal names, timing, and availability of a retirement pulse, branch-outcome indication, and load/store-valid signals are core-specific. Section 6.5 and Table 12 list the signals this document assumes; these must be confirmed and, if necessary, remapped against the actual port list of the faculty-provided core before RTL integration begins, exactly as the memory map itself is described as preliminary in the original project abstract.*

2.3 Hardware Performance Monitoring Concepts

A hardware performance monitor is a small, purpose-built counter block that increments in response to a hardware condition rather than a software poll. Three counter classes are used in this project:

- **Free-running cycle counter** — increments once per enabled clock cycle while the counter is enabled; the classic building block for measuring elapsed execution time of a code region in cycles.
- **Event-completion counter** — increments once per occurrence of a discrete hardware event, such as an instruction retiring; used for the Instruction Counter.
- **Programmable event counter** — a single physical counter multiplexed across a small set of candidate events via an event-select register, trading the ability to monitor several events simultaneously for a smaller gate count; used for the Performance Event Counter.

All three share a common software control model: a control register with independent *enable* and *clear* bits, and one or more read-only count registers. This mirrors the control model of the standard RISC-V *mcountinhibit* CSR (which independently gates *mcycle* and *minstret*) but implemented as ordinary memory-mapped registers rather than CSRs, so that any bus master — not only the hart executing CSR instructions — can configure and read them.

2.4 Memory-Mapped System Bus Protocol

The SoC uses a single, synchronous, memory-mapped system bus connecting the processor's load/store interface to Instruction Memory, Data Memory, and every peripheral, including the three monitoring IPs. The bus is intentionally simple: a single bus master (the processor's load/store port) drives address, write data, and control signals; the addressed slave drives read data and a ready/acknowledge signal one or more cycles later. This keeps the additional RTL small and avoids the complexity of a multi-master crossbar, matching a project whose primary RTL contribution is the three monitoring IPs and the surrounding integration rather than the bus fabric itself.

***Note:** The bus protocol described in Chapter 6 is a lightweight, Wishbone/APB-style valid/ready handshake chosen for its simplicity and ease of verification. If the faculty-provided core exposes a different native bus protocol (for example, a simple single-cycle memory interface, AHB-Lite, or a custom interface), a thin protocol adapter should be added between the processor's load/store port and the system bus described here; this does not change the memory map, register definitions, or monitoring IP behaviour.*

3. Proposed SoC Architecture

3.1 System Overview

The SoC is organised around the provided RISC-V processor core. Instruction Memory stores the application program; Data Memory stores program data. A bus interconnect performs address decoding and connects the processor to the mandatory peripherals (UART, Timer, GPIO) and the three additional performance-monitoring IP blocks (Cycle Counter, Instruction Counter, Performance Event Counter) over the shared memory-mapped system bus described in Chapter 6.

The processor fetches and executes a small test application from Instruction Memory. Before running the timed code region, the application clears and enables the three monitoring counters through their control registers. After the region completes, the application disables the counters, reads the accumulated cycle count, instruction count, and selected event count, formats the results, and transmits them to a host or simulation console via UART. GPIO is driven with a simple completion indicator at the end of the run.

3.2 Block Diagram

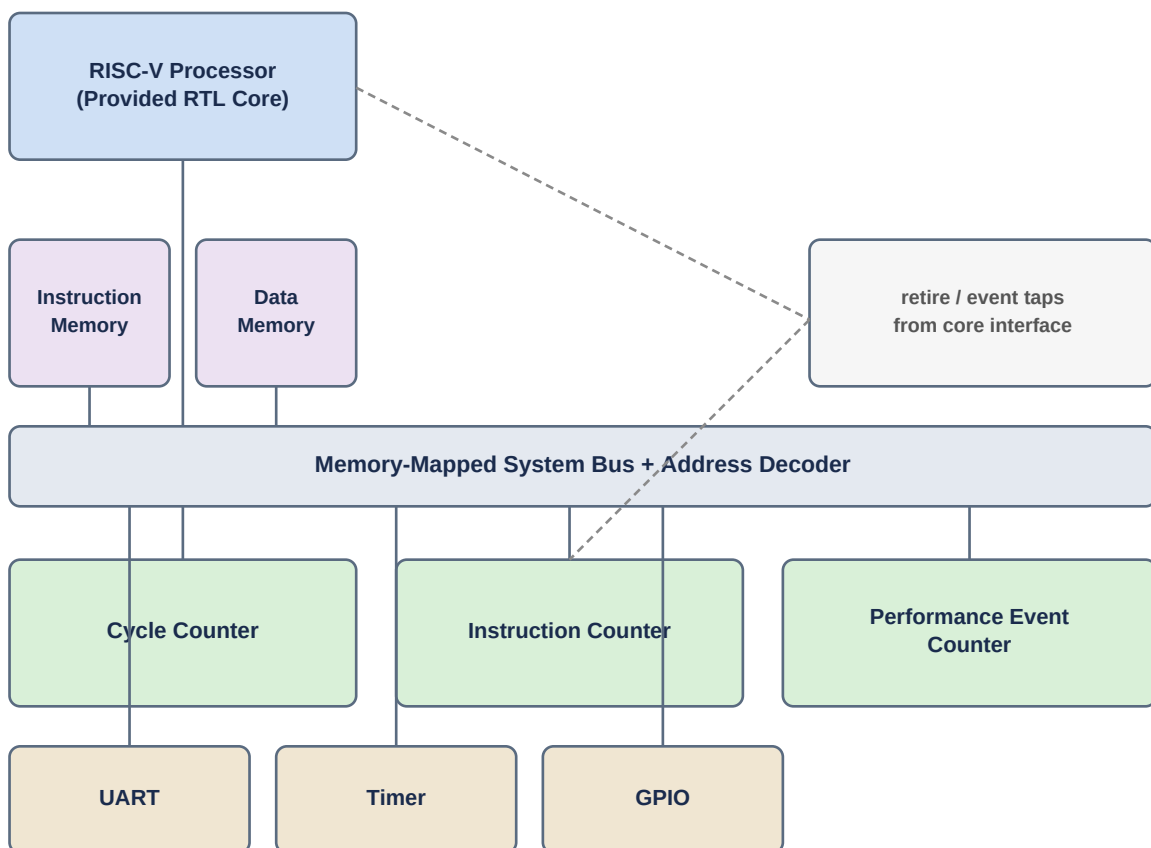


Figure 1: SoC Block Diagram — RISC-V Core, Memory-Mapped System Bus, Memories, Monitoring IPs, and Mandatory Peripherals.

The processor connects to the bus through a single load/store interface. Instruction Memory and Data Memory sit on the same bus as the peripherals and monitoring IPs, decoded by address range as listed in

Chapter 5. The dashed lines in Figure 1 indicate the point-to-point retirement/event signals tapped directly from the processor's interface into the Instruction Counter and Performance Event Counter, in addition to each block's ordinary AXI/bus-style control-register interface — these are control/observation taps, not bus transactions.

3.3 Block Summary

Block	Role	Interface
RISC-V Processor Core	Fetches and executes the application; configures and reads all monitoring and peripheral registers.	Bus master (instr. fetch + load/store)
Instruction Memory	Stores the application program.	Bus slave (read, or read/write for loading)
Data Memory	Stores program data and the application's working state.	Bus slave (read/write)
System Bus / Address Decoder	Routes processor accesses to the addressed slave.	Single-master interconnect
Cycle Counter	Counts enabled processor clock cycles.	Bus slave + core clock-enable tap
Instruction Counter	Counts instruction-retirement events from the processor interface.	Bus slave + core retire-pulse tap
Performance Event Counter	Counts one selected processor/system event chosen by an event-select register.	Bus slave + core event taps
UART	Serial reporting of measured performance values to host/console.	Bus slave
Timer	Baseline timing peripheral; may schedule test runs.	Bus slave
GPIO	Simple completion/status indicator.	Bus slave

Table 1: SoC Block Summary

4. Detailed Description of Each SoC Block

4.1 RISC-V Processor Core

The provided RISC-V processor core is the sole processing element in the SoC. It fetches and executes the test application from Instruction Memory, accesses Data Memory and every memory-mapped peripheral through its load/store interface, and is responsible for configuring (enable/clear/event-select) and reading back all three monitoring IPs. The core is used unmodified; the monitoring IPs are designed to attach to whatever retirement and event signals the core already exposes at its boundary, rather than requiring changes inside the core's pipeline.

4.2 Instruction Memory

Instruction Memory holds the compiled test application. It is accessed by the processor's instruction-fetch path every cycle during normal execution and, for simulation purposes, may additionally be loaded from a memory-initialisation file (hex/mem) prior to reset release, which is the conventional approach for RTL simulation without a bootloader.

4.3 Data Memory

Data Memory holds the application's working data — variables, buffers used by the computational test loop, and formatted result strings staged for UART transmission. It is accessed through the processor's load/store interface over the shared system bus.

4.4 System Bus and Address Decoder

The system bus is a single-master, synchronous, memory-mapped interconnect. It decodes the processor's address on every load/store transaction and routes the transaction to the addressed slave: Instruction Memory, Data Memory, UART, Timer, GPIO, or one of the three monitoring IPs. Because there is exactly one bus master in this SoC, no arbitration logic is required; the decoder is a straightforward combinational address comparator driving the select and ready signals described in Chapter 6.

4.5 Cycle Counter

The Cycle Counter is a free-running, 64-bit hardware counter (implemented as two 32-bit memory-mapped registers) that increments once for every clock cycle in which the counter is enabled. It contains a control register with independent enable and clear bits and a pair of read-only count registers. Because it counts clock cycles directly rather than a processor-internal event, it provides a simple, direct hardware measurement of the elapsed execution time — in clock cycles — of a program or a selected code region, without depending on any core-internal signal beyond the shared system clock and its own enable bit.

4.6 Instruction Counter

The Instruction Counter is a 64-bit hardware counter that increments on every instruction-completion (retirement) pulse supplied by the processor interface. It shares the same control-register model as the Cycle Counter (enable/clear, plus a pair of read-only count registers) and provides a bus-visible equivalent of the RISC-V minstret CSR, readable by any bus master rather than only via a CSR instruction executed by the hart. The precise retirement signal (a single-cycle 'instruction valid and completing' pulse, or an

equivalent core-provided indication) will be selected according to the signals actually available on the faculty-provided RISC-V RTL, as noted in Section 2.2.

4.7 Performance Event Counter

The Performance Event Counter is a single, programmable 32-bit counter that tracks one selected processor or system event at a time, chosen via a small event-select register (Table 8). Depending on the signals exposed by the provided RISC-V core, monitorable events may include retired branch instructions, mispredicted branches, retired load/store instructions, or bus wait/stall cycles observed on the memory interface. The counter shares the same enable/clear control model as the other two counters, plus the additional event-select field. Only one event is counted at a time in this design; monitoring several events concurrently would require either replicating the counter or adding multiplexed accumulation banks, both of which are noted as future enhancements in Chapter 12.

4.8 UART

The UART provides byte-oriented serial communication between the SoC and an external host or the simulation console. Firmware formats the cycle, instruction, and event counts as text and writes them byte-by-byte to the UART transmit register once the test application's monitored region completes.

4.9 Timer

The Timer is a standard free-running/reload countdown peripheral. In this project it serves as a mandatory baseline peripheral; it may optionally be used by firmware to bound or schedule the test application's execution window, but it is not required by the monitoring IPs themselves.

4.10 GPIO

GPIO provides a minimal external interface: an output can be driven as a simple pass/complete indicator once the test application finishes reporting its results, and, optionally, input pins can be used to gate when the monitored region begins.

5. Memory Subsystem and Memory Map

5.1 Overview

All addressable blocks in the SoC — Instruction Memory, Data Memory, the mandatory peripherals, and the three monitoring IPs — share a single flat 32-bit address space. Each peripheral and monitoring IP is allocated a 256-byte (0x100) aligned region, which is more than sufficient for the small register sets defined in Chapter 7 and leaves headroom for future registers without renumbering neighbouring blocks.

5.2 Top-Level Memory Map

Address Range	Module	Size
0x0000_0000 – 0x0000_0FFF	Instruction Memory	4 KB
0x0000_1000 – 0x0000_1FFF	Data Memory	4 KB
0x0000_2000 – 0x0000_20FF	UART	256 B
0x0000_2100 – 0x0000_21FF	Timer	256 B
0x0000_2200 – 0x0000_22FF	GPIO	256 B
0x0000_2300 – 0x0000_23FF	Cycle Counter	256 B
0x0000_2400 – 0x0000_24FF	Instruction Counter	256 B
0x0000_2500 – 0x0000_25FF	Performance Event Counter	256 B
0x0000_2600 – 0x0000_FFFF	Reserved / peripheral expansion	—

Table 2: Top-Level Memory Map

Note: These address ranges are preliminary, carried over directly from the project abstract, and may be adjusted once the bus interface and address-decoding requirements of the faculty-provided RISC-V processor are examined in detail (for example, if the core's reset vector or default memory regions constrain where Instruction Memory must be placed).

5.3 Memory Type and Access Properties

Memory / Region	Read/Write	Notes
Instruction Memory	Read (fetch); write for program load in simulation	Holds the application binary
Data Memory	Read/Write	Holds application data and result buffers
Peripheral / monitoring control-status registers	Read/Write (register-dependent)	Non-idempotent; some fields are write-1-to-clear

Table 3a: Memory Type and Access Properties

6. Bus and Interconnect

6.1 Bus Protocol Overview

The system bus uses a simple, synchronous, single-outstanding-transaction valid/ready handshake. The processor (bus master) drives an address, a write-enable qualifier, write data, and a valid strobe; the addressed slave decodes its own chip-select internally from the address, and returns read data (for reads) together with a ready acknowledgement, either combinationally or after a small, fixed number of wait cycles. This is deliberately simpler than a full split-transaction protocol such as AXI4, which is unnecessary for a single-master SoC whose principal RTL contribution is the three monitoring IPs rather than the bus fabric.

6.2 System Bus Signals

Signal	Driven By	Width	Description
clk	SoC	1	System clock, common to processor, bus, and all slaves
rst_n	SoC	1	Active-low synchronous reset
bus_addr[31:0]	Master	32	Byte address of the current transaction
bus_wdata[31:0]	Master	32	Write data
bus_wstrb[3:0]	Master	4	Per-byte write strobes
bus_we	Master	1	Write enable (1 = write, 0 = read)
bus_valid	Master	1	Asserted while a transaction request is active
bus_rdata[31:0]	Slave	32	Read data returned by the addressed slave
bus_ready	Slave	1	Asserted when the addressed slave has completed the transaction
bus_sel[7:0]	Decoder	8	One-hot slave select, generated from bus_addr

Table 4: System Bus Signals — Signal Breakdown

6.3 Address Decoding

The address decoder is a combinational block that compares `bus_addr` against the ranges of Table 2 and asserts exactly one bit of `bus_sel`. Read data and ready are then multiplexed back to the master from the selected slave. An access to an unmapped address (outside every range in Table 2) returns `bus_ready` with `bus_rdata` driven to zero after one cycle, avoiding a bus hang; a future revision could instead register a decode-error status bit, as noted in Chapter 12.

6.4 Arbitration

This SoC has a single bus master — the processor's load/store interface. No arbitration logic is required. If a future revision adds a second master (for example, a debug or DMA-style access path directly into the monitoring counters), a simple fixed-priority or round-robin arbiter would need to be added ahead of the address decoder; this is out of scope for the present design.

6.5 Core-Facing Signals Assumed by the Monitoring IPs

In addition to the ordinary system-bus register interface each monitoring IP exposes as a bus slave, the Instruction Counter and Performance Event Counter also need one or more point-to-point signals tapped directly from the processor core's boundary, since these events occur inside the pipeline and are not bus transactions. The Cycle Counter needs only the shared system clock and, optionally, a processor-driven clock-enable/halt indication so that cycle counting can be gated while the core is stalled or halted, if desired.

Signal (assumed)	Direction	Width	Used By	Description
clk	Core → SoC (shared)	1	Cycle Counter	Common system clock; counted directly
core_active / core_clk_en	Core → Cycle Counter	1	Cycle Counter (optional)	Gates cycle counting while the core is halted/stalled, if the core exposes such a signal
instr_retire	Core → Instruction Counter	1	Instruction Counter	Single-cycle pulse asserted when an instruction completes/retires
branch_retire	Core → Performance Event Counter	1	Event Counter (event 0)	Pulse on a retired branch instruction
branch_mispredict	Core → Performance Event Counter	1	Event Counter (event 1)	Pulse on a mispredicted branch, if exposed
mem_access_valid	Core → Performance Event Counter	1	Event Counter (events 2–3)	Pulse on a completed load or store
bus_wait / mem_stall	Bus / Core → Performance Event Counter	1	Event Counter (events 4–5)	Asserted while a bus or memory access is stalling the pipeline

Table 12: Core-Facing Bus/Event Signals (assumed; to be finalized against the faculty-provided core's actual port list)

Note: None of these core-facing signal names are guaranteed to exist verbatim on the provided RTL. Before integration, each row of Table 12 must be matched against the real core's port list (or derived combinationally from signals that are exposed), exactly as the original project abstract notes that 'the exact event connection will be selected according to the signals available in the faculty-provided RISC-V RTL.'

7. Register-Level Interface

7.1 Cycle Counter Registers — Base Address 0x0000_2300

Offset	Register	Access	Description
0x00	CYCLE_CTRL	R/W	Bit 0: ENABLE — counts while set. Bit 1: CLEAR (W1, self-clearing) — resets the count to zero. Bits [31:2]: reserved.
0x04	CYCLE_COUNT_LO[31:0]	RO	Low 32 bits of the accumulated cycle count.
0x08	CYCLE_COUNT_HI[31:0]	RO	High 32 bits of the accumulated cycle count (64-bit counter, mirroring mcycle/mcycleh).
0x0C	Reserved	—	Reserved for future use.

Table 5: Cycle Counter Registers

7.2 Instruction Counter Registers — Base Address 0x0000_2400

Offset	Register	Access	Description
0x00	INSTR_CTRL	R/W	Bit 0: ENABLE — counts while set. Bit 1: CLEAR (W1, self-clearing). Bits [31:2]: reserved.
0x04	INSTR_COUNT_LO[31:0]	RO	Low 32 bits of the accumulated retired-instruction count.
0x08	INSTR_COUNT_HI[31:0]	RO	High 32 bits of the accumulated retired-instruction count (64-bit counter, mirroring minstret/minstreth).
0x0C	Reserved	—	Reserved for future use.

Table 6: Instruction Counter Registers

7.3 Performance Event Counter Registers — Base Address 0x0000_2500

Offset	Register	Access	Description
0x00	EVENT_CTRL	R/W	Bit 0: ENABLE — counts while set. Bit 1: CLEAR (W1, self-clearing). Bits [31:2]: reserved.
0x04	EVENT_SELECT[2:0]	R/W	Selects the monitored event per Table 8. Must be written while ENABLE = 0 to avoid a mixed-event count during the transition.
0x08	EVENT_COUNT[31:0]	RO	Accumulated count of the currently selected event (32-bit; single events are expected to occur far less often than clock cycles, so a 64-bit counter is not required).
0x0C	Reserved	—	Reserved for future use.

Table 7: Performance Event Counter Registers

7.4 Event Select Encoding

EVENT_SELECT	Event	Signal Basis (Table 12)
0b000	Retired branch instructions	branch_retire

EVENT_SELECT	Event	Signal Basis (Table 12)
0b001	Mispredicted branches	branch_mispredict
0b010	Retired load instructions	mem_access_valid (load qualifier)
0b011	Retired store instructions	mem_access_valid (store qualifier)
0b100	Memory-access bus wait/stall cycles	bus_wait / mem_stall
0b101 – 0b111	Reserved	—

Table 8: Event Select Encoding

Note: The exact set of monitorable events depends on which signals the faculty-provided core actually exposes at its boundary (Section 6.5). Events that cannot be sourced directly should be removed or replaced, and EVENT_SELECT re-documented accordingly, before RTL is finalized.

7.5 UART Registers — Base Address 0x0000_2000

Offset	Register	Access	Description
0x00	UART_BAUD_DIV[15:0]	R/W	Baud-rate divisor
0x04	UART_TX_DATA[7:0]	WO	Byte to transmit
0x08	UART_RX_DATA[7:0]	RO	Last received byte
0x0C	UART_STATUS	RO	Bit 0: TX_BUSY. Bit 1: RX_VALID. Bit 2: FRAMING_ERR

Table 9: UART Registers

7.6 Timer Registers — Base Address 0x0000_2100

Offset	Register	Access	Description
0x00	TIMER_EN	R/W	Bit 0: enables the countdown
0x04	TIMER_RELOAD[31:0]	R/W	Reload/count value, in clock cycles
0x08	TIMER_COUNT[31:0]	RO	Current countdown value (diagnostic read)
0x0C	TIMER_STATUS	RO	Bit 0: TICK — pulses on expiry (write-1-to-clear)

Table 10: Timer Registers

7.7 GPIO Registers — Base Address 0x0000_2200

Offset	Register	Access	Description
0x00	GPIO_DIR[7:0]	R/W	Direction register: 1 = output
0x04	GPIO_OUT[7:0]	R/W	Output drive value
0x08	GPIO_IN[7:0]	RO	Captured external input state

Table 11: GPIO Registers

8. Control Flow

8.1 Overview

The control flow of the SoC describes how firmware configures, starts, and reads back the monitoring counters around a timed application region, and how the mandatory peripherals are initialised at boot. There are three sub-paths: boot/initialisation, counter arm-and-clear, and post-execution readout/reporting.

8.2 Boot and Initialisation Sequence

- 1 After reset de-assertion, the processor fetches its first instruction from Instruction Memory at the core's reset vector.
- 2 Configure GPIO: set the direction register (GPIO_DIR) and clear outputs.
- 3 Configure UART: write UART_BAUD_DIV; the UART is otherwise always enabled in this design.
- 4 Configure Timer, if used for scheduling: write TIMER_RELOAD and set TIMER_EN.
- 5 Configure the Performance Event Counter's EVENT_SELECT field while its ENABLE bit is still 0.
- 6 Begin the main application loop.

8.3 Counter Arm-and-Clear Sequence (Start of Monitored Region)

- 1 Write CLEAR = 1 to CYCLE_CTRL, INSTR_CTRL, and EVENT_CTRL (each self-clears after resetting its count register(s) to zero).
- 2 Write ENABLE = 1 to CYCLE_CTRL, INSTR_CTRL, and EVENT_CTRL, arming all three counters together so that they cover an identical window of execution.
- 3 Execute the computational task or loop being measured.

8.4 Readout and Reporting Sequence (End of Monitored Region)

- 1 Write ENABLE = 0 to CYCLE_CTRL, INSTR_CTRL, and EVENT_CTRL, freezing all three counters.
- 2 Read CYCLE_COUNT_LO/HI, INSTR_COUNT_LO/HI, and EVENT_COUNT.
- 3 Format the three values as text.
- 4 Transmit the formatted results byte-by-byte via UART_TX_DATA, polling UART_STATUS.TX_BUSY before each byte.
- 5 Drive GPIO_OUT with a simple completion indicator.

Note: Disabling all three counters together, immediately before the register reads, avoids any possibility of the counts advancing between the end of the monitored region and the actual readout, which would otherwise slightly overstate the measured cycle/instruction/event totals.

8.5 Optional Threshold/Status Reporting

None of the three monitoring IPs generate an interrupt in the baseline design; they are read by polling at the end of the monitored region, consistent with a small demonstration application. A future revision could add a programmable threshold-compare and interrupt output on any of the three counters, as discussed in Chapter 12.

9. Data Flow

9.1 Overview

The SoC has four distinct data paths relevant to performance monitoring, in addition to the processor's ordinary instruction-fetch and load/store paths.

9.2 Cycle Counting Path

```
CYCLE_CTRL.ENABLE = 1
|
v (every clk edge while ENABLE = 1)
CYCLE_COUNT[63:0]++
|
v
CPU reads CYCLE_COUNT_LO / CYCLE_COUNT_HI over the system bus
```

9.3 Instruction-Retirement Counting Path

```
Processor core executes instructions
|
v (instr_retire pulse asserted on completion, per instruction)
INSTR_CTRL.ENABLE = 1 gates the increment
|
v
INSTR_COUNT[63:0]++
|
v
CPU reads INSTR_COUNT_LO / INSTR_COUNT_HI over the system bus
```

9.4 Event Counting Path

```
Firmware writes EVENT_SELECT (Table 8) while EVENT_CTRL.ENABLE = 0
|
v
EVENT_CTRL.ENABLE = 1
|
v (per occurrence of the selected event signal, Table 12)
EVENT_COUNT[31:0]++
|
v
CPU reads EVENT_COUNT over the system bus
```

9.5 UART Reporting Path

```
Firmware result processing (end of monitored region):
read CYCLE_COUNT_LO/HI, INSTR_COUNT_LO/HI, EVENT_COUNT
|
v
format result bytes (ASCII)
|
v
write UART_TX_DATA, poll UART_STATUS.TX_BUSY --> uart_tx --> host / simulation console
|
v
```

```
write GPIO_OUT (completion indicator) --> gpio_out[7:0]
```

10. Design Methodology

10.1 Overview

The design follows a top-down hardware/software co-design methodology: the system architecture is defined first at the block and interface level (Chapters 3–7), followed by RTL implementation of each block, integration on the shared system bus, and simulation-based verification (Chapter 11). All RTL is written in synthesisable SystemVerilog.

10.2 RTL Coding Guidelines

- Explicit signed/unsigned declarations and casts for all arithmetic operations.
- Explicit bit widths for all registers, wires, and arithmetic intermediates (in particular for the 64-bit cycle and instruction counters, which are built from two concatenated 32-bit registers rather than a single 64-bit register, to keep every register bus-word aligned).
- No floating-point in synthesisable logic.
- No # delays in synthesisable RTL.
- No testbench-only constructs in synthesisable modules.
- A single synchronous, active-low reset (rst_n) used consistently across every block.
- No vendor-specific primitives in the synthesisable core RTL.

10.3 Block-Level Implementation Order

Stage	Block	Key Implementation Notes
1	Processor core integration/wrapper	Connect the provided core's fetch and load/store interfaces to the system bus; identify and tap the retirement/event signals of Table 12.
2	System Bus and Address Decoder	Single-master valid/ready interconnect; address decode per Table 2; unmapped-access handling.
3	Instruction Memory and Data Memory	Instantiate at their mapped base addresses; add simulation memory-initialisation support.
4	Cycle Counter	64-bit free-running counter; enable/clear control register; clock gating for the count logic.
5	Instruction Counter	64-bit event counter; retirement-pulse synchronisation (if the tapped signal crosses any boundary); enable/clear control register.
6	Performance Event Counter	32-bit counter; event-select multiplexer; enable/clear control register.
7	UART	Baud-rate divisor; TX shift register; RX shift register; status flags.
8	Timer	Reload/countdown register; tick generation.
9	GPIO	Direction register; output register; input capture.

Stage	Block	Key Implementation Notes
10	Top-level integration	Connect every block to the bus and to the core's point-to-point taps; verify the complete memory map.

Table 13: RTL Implementation Order

10.4 Firmware Design

Firmware is written in C targeting the RV32I(M) ABI matching the provided core. Its responsibilities are: system initialisation (Section 8.2), the counter arm-and-clear/readout sequence around the monitored computational task (Sections 8.3–8.4), UART formatting and transmission of results, and GPIO completion signalling. Peripheral and monitoring-IP registers are accessed through standard memory-mapped volatile pointer dereferences; no CSR instructions are required for reading the monitoring IPs, since — unlike mcycle/minstret — they are ordinary memory-mapped registers.

11. Verification and Test Plan

11.1 Verification Strategy

The verification strategy follows a hierarchical approach: each block (system bus/decoder, Cycle Counter, Instruction Counter, Performance Event Counter, UART, Timer, GPIO) is first verified independently with a dedicated unit testbench, then the blocks are integrated progressively and tested at the SoC level with the processor executing the test application. All test results described in this chapter are planned/expected; actual pass/fail results are to be recorded upon completion of simulation runs.

11.2 Testbench Architecture

```
tb_soc_top
|-- clock/reset generation
|-- RISC-V processor core (provided RTL, loaded with test-application image)
|-- System bus monitor (protocol/handshake checking)
|-- Cycle Counter (DUT)
|-- Instruction Counter (DUT)
|-- Performance Event Counter (DUT)
|-- UART (DUT) + UART monitor
|-- Timer (DUT)
|-- GPIO (DUT) + GPIO monitor
|-- System Bus / Address Decoder (DUT)
|-- Instruction Memory / Data Memory (behavioural or RTL models)
|-- Event-signal stimulus generator (drives instr_retire, branch, memory-access
|   taps directly for unit-level testing of the counters, independent of the core)
|-- Assertions (bus protocol rules, counter enable/clear behaviour)
`-- Scoreboards / checkers
```

Figure 2 (planned): Testbench Architecture for Full-SoC Simulation.

11.3 Unit Test Plan

Block	Test Coverage	Status
System Bus / Address Decoder	Address decode correctness; valid/ready handshake compliance; unmapped address returns zero read data with ready asserted.	Planned
Cycle Counter	Increments exactly once per clock while ENABLE = 1; holds while ENABLE = 0; CLEAR resets both count registers to zero on the following cycle; 32-bit rollover from COUNT_LO into COUNT_HI.	Planned
Instruction Counter	Increments exactly once per instr_retire pulse while ENABLE = 1; back-to-back pulses on consecutive cycles are each counted; CLEAR and ENABLE behaviour matches the Cycle Counter.	Planned
Performance Event Counter	EVENT_SELECT correctly multiplexes each event source in Table 8; count increments only for the selected event; switching EVENT_SELECT while ENABLE = 0 does not corrupt the existing count; CLEAR/ENABLE behaviour.	Planned
UART	TX serialisation; RX deserialisation; baud-rate configuration; framing-error detection; busy/valid status flags.	Planned

Block	Test Coverage	Status
Timer	Reload/countdown accuracy; tick generation at the programmed interval.	Planned
GPIO	Direction register; output drive; input capture.	Planned

Table 14: Unit Test Plan Summary

11.4 Integration Test Plan

Test Name	Description	Status
CPU-to-Peripheral Access	Processor reads/writes every mapped register in Chapter 7 over the system bus; response completes with the expected data retained.	Planned
Unmapped Access	Access to an unmapped address completes without a bus hang; read data is zero.	Planned
Cycle Count Accuracy	Run a fixed-length instruction sequence with a known, deterministic cycle count (no branches/stalls); compare CYCLE_COUNT against the expected value.	Planned
Instruction Count Accuracy	Run a fixed-length instruction sequence with a known instruction count; compare INSTR_COUNT against the expected value.	Planned
Event Count — Branches	Run a loop with a known number of taken/not-taken branches; select EVENT_SELECT = branch-retired; compare EVENT_COUNT against the expected count.	Planned
Event Count — Loads/Stores	Run a loop with a known number of load/store instructions; select the corresponding EVENT_SELECT value; compare EVENT_COUNT against the expected count.	Planned
Counter Isolation	Verify that disabling/clearing one counter (e.g., the Performance Event Counter) does not perturb the Cycle Counter or Instruction Counter running concurrently.	Planned
Full Application Run	Test application runs end-to-end: clears/enables counters, executes a known computational task, disables counters, reads results, and reports them via UART; verify UART output matches the expected cycle/instruction/event values.	Planned

Table 15: Integration Test Plan

11.5 Expected Results

Note: No actual simulation results, waveforms, or coverage numbers are available at the time of writing. All results in this section are expected/planned outcomes based on the architectural specification and must be updated with actual results upon completion of verification.

- **Cycle Counter:** for a test region of N clock cycles with the counter enabled throughout, CYCLE_COUNT = N exactly, with no under- or over-count at the enable/disable boundary given the arm-and-clear/readout sequence of Section 8.3–8.4.
- **Instruction Counter:** for a test region retiring M instructions, INSTR_COUNT = M exactly, matching the count that minstret would report over the same window, if the core implements it.

- **Performance Event Counter:** for a test region containing a known number of occurrences K of the selected event, `EVENT_COUNT` = K exactly; switching `EVENT_SELECT` between runs and re-clearing produces the corresponding count for the newly selected event with no residue from the previous selection.
- **UART Reporting:** the transmitted byte stream, decoded, matches the three counter values read by firmware immediately after the monitored region.

11.6 Assertions

- **Bus:** `bus_valid` remains asserted until the corresponding `bus_ready` handshake is received.
- **Cycle Counter:** the count register never changes while `ENABLE` = 0 and `CLEAR` is not asserted.
- **Instruction Counter:** exactly one increment occurs per `instr_retire` pulse, with no missed or double-counted pulses across a clock-domain or pipeline boundary.
- **Performance Event Counter:** `EVENT_COUNT` only changes for the event indicated by the current `EVENT_SELECT` value.
- **All three counters:** `CLEAR` (W1) always brings the associated count register(s) to zero on the next cycle, regardless of the current `ENABLE` state.

12. Limitations and Future Enhancements

12.1 Current Limitations

12.1.1 Single Event Monitored at a Time

The Performance Event Counter tracks only one selected event at a time. Comparing two events over the same execution window requires two separate runs of the test application, one per `EVENT_SELECT` value, rather than a single simultaneous measurement.

12.1.2 Dependence on the Provided Core's Exposed Signals

The Instruction Counter and every configurable event in Table 8 depend on the faculty-provided RISC-V core exposing a usable retirement pulse and the corresponding event-qualifier signals at its boundary. If a given signal is not available, the associated event must be removed from Table 8 or derived indirectly (for example, from bus activity rather than a core-internal signal), which may reduce measurement precision.

12.1.3 No Interrupt-Driven Reporting

Counters are read by polling at the end of a monitored region rather than via a threshold-triggered interrupt. Long-running or continuously monitored applications would need software to periodically stop and read the counters, or a future threshold/interrupt enhancement (Section 12.2.3).

12.1.4 No Physical Synthesis Timing Closure

This project targets technology-independent synthesisable RTL, verified at RTL simulation level. Physical timing closure, area optimisation, and power analysis at a specific technology node are not performed.

12.1.5 No Overflow Handling Beyond Natural 64-bit / 32-bit Rollover

The Cycle Counter and Instruction Counter are 64-bit and are not expected to overflow within any realistic simulation run; no overflow interrupt or sticky overflow flag is implemented. The 32-bit Performance Event Counter could overflow for very long-running, high-frequency events (such as memory-stall cycles) and silently wraps in the baseline design.

12.2 Future Enhancements

12.2.1 Multi-Event Concurrent Counting

A future version could replicate the Performance Event Counter into several banks (matching the RISC-V `mhpmcounter3–31 / mhpmevent3–31` model), each with its own `EVENT_SELECT`, allowing several events to be measured concurrently within a single run.

12.2.2 Threshold Interrupts

Each counter could be extended with a programmable threshold-compare register and an interrupt output, allowing firmware to be notified asynchronously rather than polling, and enabling continuous/background performance monitoring during normal operation rather than only around a bounded test region.

12.2.3 Sticky Overflow Flags

A write-1-to-clear overflow flag could be added to each counter's control/status register, set whenever the count register wraps, giving software a reliable way to detect an overflow that occurred between two polled reads.

12.2.4 Region-Scoped (Start/Stop Pair) Measurement Helper

A small hardware helper that automatically clears and enables all three counters on a single write, and disables all three on a second write, would reduce the four to six register writes in Sections 8.3–8.4 to two, and guarantee the counters share an identical start/stop boundary without relying on firmware ordering.

12.2.5 Decode-Error Status Bit

Rather than silently returning zero on an unmapped-address access, the address decoder could register a sticky decode-error status bit readable by firmware, improving debuggability during bring-up.

13. Conclusion

This project has designed and specified a RISC-V based SoC for hardware performance monitoring and event analysis, built around a faculty-provided RISC-V processor core used without modification. The system addresses a specific practical limitation of software-only performance measurement: the standard RISC-V mcycle and minstret CSRs are accessible only to the hart itself via CSR instructions, adding execution overhead and offering no bus-level visibility, and they do not standardize any mechanism for counting additional, project-selected events.

The key architectural contribution is a set of three small, independently controllable, memory-mapped counter IPs — a Cycle Counter, an Instruction Counter, and a Performance Event Counter — attached to the processor's existing bus and to a small number of point-to-point signals tapped from the processor's boundary, requiring no modification to the provided core's internal RTL. A shared enable/clear control model across all three counters allows firmware to arm, run, and read back a consistent measurement window with a short, well-defined register sequence.

The design is specified at the RTL architecture level with a complete memory map, register-level interface for every block, control- and data-flow descriptions, and a detailed verification plan covering both unit-level and full-SoC integration testing. Actual simulation results, waveforms, and coverage data are to be completed during the implementation and verification phase of the project, once the exact signal set exposed by the faculty-provided RISC-V core (Section 2.2 and Table 12) has been confirmed.

In summary, the proposed SoC demonstrates that direct, bus-visible hardware performance monitoring — complementary to, but independent of, the ISA-mandated CSR-based counters — can be achieved with a small, modular set of custom IPs, a straightforward memory-mapped system bus, and a disciplined arm/run/read firmware sequence, while satisfying the project requirement of at least three additional IP blocks, a complete memory map, an application running on the processor, and simulation-based functional verification.

References

- [1] RISC-V International, "The RISC-V Instruction Set Manual, Volume I: Unprivileged ISA," Version 20191213, 2019.
- [2] RISC-V International, "The RISC-V Instruction Set Manual, Volume II: Privileged Architecture," Version 20190608-Priv-MSU-Ratified, 2019.
- [3] RISC-V International, "RISC-V External Debug Support," Version 0.13.2, 2019.
- [4] Faculty-provided RISC-V RTL processor core — datasheet / RTL documentation (to be cited once the specific core and revision are confirmed).
- [5] D. Patterson and A. Waterman, "The RISC-V Reader: An Open Architecture Atlas," Strawberry Canyon, 2017.
- [6] ARM Limited / relevant bus reference material, for comparison of memory-mapped bus handshake conventions (AMBA APB/AHB-Lite specification), as design background for the system-bus protocol described in Chapter 6.